CS452: Intro to DL Internals

Assignment 1: The Building Blocks - Scalars and Operations

Instructor: Ram Vikas, PhD

Objective

In this first session, we will understand the most fundamental unit of a neural network: a number that knows its own history. We'll start by seeing how PyTorch handles this with its `Tensor` objects and then build our own simple version, a `Scalar` class, from scratch. The goal is to understand that a neural network is just a big mathematical expression, and to compute gradients, we need to remember how the final result was constructed.

Part A: Warm-up with PyTorch (Approx. 30 minutes)

Let's see the "magic" of PyTorch first. We will create two numbers, ask PyTorch to track their operations, perform some math, and see how it automatically calculates their influence (gradient) on the final result.

Task 1: Create Tensors and Track Gradients Your task is to write a Python script that performs the following steps:

a. Import the `torch` library.

b. Create three `torch.tensor` objects named `a`, `b`, and `c` with floating-point values `2.0`, `3.0`, and `5.0` respectively.

c. When creating them, you must set the argument `requires_grad=True`. This is the key that tells PyTorch to build a computation graph.

d. Perform the mathematical operation `L = a * b + c`.

e. Print the resulting value of `L`. You can access the raw number using `L.item()`.

f. Call the `backward()` method on the final result, `L`. This is the command that triggers the gradient calculation.

g. Print the gradients of `a`, `b`, and `c` by accessing their `.grad` attribute (e.g., `a.grad.item()`).

h. **Think:** What do these gradient values represent? Why is `a.grad` equal to the value of `b`?

Part B: Building our own 'Scalar' Class (Approx. 90 minutes)

Now, let's demystify the magic. We will create a Python class named `Scalar` that will hold a single number but will also remember how it was created. This is the first step towards building our own auto-differentiation engine.

Task 2: The 'Scalar' Class Blueprint Create a class named `Scalar`.

a. The `__init__` method should accept a single argument, `data`.

b. Inside `__init__`, store the `data`. Also, initialize three more attributes:

- `grad`: This should be initialized to `0.0`. It will store the gradient.

- `_prev`: An empty set that will later store the "parent" `Scalar` objects.
- `_op`: An empty string that will store the operation that created this `Scalar`.

c. Implement the `__repr__` method to return a user-friendly string representation, such as `"Scalar(data=2.0)"`.

Task 3: Implementing Addition and Multiplication We need to teach our `Scalar` objects how to interact. When we compute `a + b`, we want a *new* `Scalar` object that knows it was made from `a` and `b` through addition.

a. Implement the special method `__add__(self, other)`. Inside this method:

- Create a new `Scalar` object, `out`.
- The data for `out` should be `self.data + other.data`.
- Set the `_prev` attribute of `out` to be a set containing its parents: {`self, other`}.
- Set the `_op` attribute of `out` to be the string `'+'`.
- Return the new `out` object.

b. Implement the `__mul__(self, other)` method following the exact same logic as addition, but for multiplication.

c. **Test your implementation:** Create three `Scalar` objects and compute `L = a * b + c`. Print `L` and then inspect its attributes `L._prev` and `L._op`. Does it correctly show that `L` was created by a '+' operation and has two parents?

Task 4: Implementing an Activation Function (ReLU) Neural networks need non-linearities. Let's implement one of the simplest: ReLU (Rectified Linear Unit), defined as `f(x) = max(0, x)`.

a. Add a new method to your `Scalar` class called `relu(self)`.

b. Inside this method, calculate the result of the ReLU function on `self.data`.

c. Create and return a *new* `Scalar` object, `out`, containing this result.

d. The parent of `out` is just `self`, so set its `_prev` to {`self`} and its `_op` to `'ReLU'`.

Conclusion

Great work! We now have a `Scalar` class that can perform basic math and build a graph of these operations by tracking its "parents". We haven't calculated any gradients yet, but we've laid the entire foundation for it. In the next lecture, we'll implement the `backward()` pass to bring this graph to life!